7
Number Theory

Number theory is perhaps the most interesting and beautiful area of mathematics. Euclid's proof that there are an infinite number of primes remains just as clear and elegant today as it was more than two thousand years ago. Innocent-looking questions like whether the equation $a^n + b^n = c^n$ has solutions for integer values of a, b, c, and $n > 2$ often turn out not to be so innocent. Indeed, this is the statement of Fermat's last theorem!

Number theory is great training in formal, rigorous reasoning, because number-theoretic proofs are clear and decisive. Studying the integers is interesting because they are such concrete and important objects. Discovering new properties of the integers is discovering something exciting about the natural world.

Computers have long been used in number theoretic research. Performing interesting number-theoretic computations on large integers requires great efficiency. Fortunately there are many clever algorithms to help us out.

7.1 Prime Numbers

A *prime number* is an integer $p > 1$ which is only divisible by 1 and itself. Said another way, if p is a prime number, then $p = a \cdot b$ for integers $a \leq b$ implies that $a = 1$ and $b = p$. The first ten prime numbers are 2, 3, 5, 7, 11, 13, 17, 19, 23, and 27.

Prime numbers are important because of the *fundamental theorem of arithmetic*. Despite the impressive title, all it states is that every integer can be expressed in only one way as the product of primes. For example, 105 is uniquely expressed as $3 \times 5 \times 7$, while 32 is uniquely expressed as $2 \times 2 \times 2 \times 2 \times 2$. This unique set of numbers multiplying

to n is called the *prime factorization* of n. Order doesn't matter in a prime factorization, so we can canonically list the numbers in sorted order. But multiplicity does; it is what distinguishes the prime factorization of 4 from 8.

We say a prime number p is a *factor* of x if it appears in its prime factorization. Any number which is not prime is said to be *composite*.

7.1.1 Finding Primes

The easiest way to test if a given number x is prime uses repeated division. Start from the smallest candidate divisor, and then try all possible divisors up from there. Since 2 is the only even prime, once we verify that x isn't even we only need try the odd numbers as candidate factors. Further, we can bless n as prime the instant we have shown that it has no non-trivial prime factors below $\sqrt{n}$. Why? Suppose not – i.e., x is composite but has a smallest non-trivial prime factor p which is greater than $\sqrt{n}$. Then x/p must also divide x, and must be larger than p, or else we would have seen it earlier. But the product of two numbers greater than $\sqrt{n}$ must be larger than n, a contradiction.

Computing the prime factorization involves not only finding the first prime factor, but stripping off all occurrences of this factor and recurring on the remaining product:

```
prime_factorization(long x)
{
        long i;                 /* counter */
        long c;                 /* remaining product to factor */

        c = x;
        while ((c % 2) == 0) {
                printf("%ld\n",2);
                c = c / 2;
        }

        i = 3;
        while (i <= (sqrt(c)+1)) {
                if ((c % i) == 0) {
                        printf("%ld\n",i);
                        c = c / i;
                }
                else
                        i = i + 2;
        }

        if (c > 1) printf("%ld\n",c);
}
```

Testing the terminating condition $i > \sqrt{c}$ is somewhat problematic, because `sqrt()` is a numerical function with imperfect precision. Just to be safe, we let i run an extra

iteration. Another approach would be to avoid floating point computation altogether and terminate when `i*i > c`. However, the multiplication might cause overflow when working on very large integers. Multiplication can be avoided by observing that $(i+1)^2 = i^2 + 2i + 1$, so adding $i + i + 1$ to i^2 yields $(i + 1)^2$.

For higher performance, we could move the `sqrt(c)` computation outside the main loop and only update it when `c` changes value. However, this program responds instantly on my computer for the prime 2,147,483,647. There exist fascinating randomized algorithms which are more efficient for testing primality on very large integers, but these are not something for us to worry about at this scale – except as the source of interesting contest problems themselves.

7.1.2 Counting Primes

How many primes are there? It makes sense that primes become rarer and rarer as we consider larger and larger numbers, but do they ever vanish? The answer is no, as shown by Euclid's proof that there are an infinite number of primes. It uses an elegant *proof by contradiction*. Knowing this proof is not strictly necessary to compete in programming contests, but it is one sign of being an educated person. Thus there is no shame in reviewing it here.

Let us assume the converse, that there are only a finite number of primes, $p_1, p_2, \ldots, p_n$. Let $m = 1 + \prod_{i=1}^{n} p_i$, i.e., the product of all of these primes, plus one. Since this is bigger than any of the primes on our list, m must be composite. Therefore some prime must divide it.

But which prime? We know that m is not divisible by p_1, because it leaves a remainder of 1. Further, m is not divisible by p_2, because it also leaves a remainder of 1. In fact, m leaves a remainder of 1 when divided by any prime p_i, for $1 \leq i \leq n$. Thus $p_1, p_2, \ldots, p_n$ cannot be the complete list of primes, because if so m must also be prime.

Since this contradicts the assumption it means there cannot exist such a complete list of primes; therefore the number of primes must be infinite! QED! [1]

Not only are there an infinite number of primes, but in fact the primes are relatively common. There are roughly $x/\ln x$ primes less than or equal to x, or put another way, roughly one out of every $\ln x$ numbers is prime.

7.2 Divisibility

Number theory is the study of integer divisibility. We say b *divides* a (denoted $b|a$) if $a = bk$ for some integer k. Equivalently, we say that b is a *divisor of* a or a is a *multiple* of b if $b|a$.

[1] Now a little puzzle to test your understanding of the proof. Suppose we take the first n primes, multiply them together, and add one. Does this number have to be prime? Give a proof or a counterexample.

As a consequence of this definition, the smallest natural divisor of every non-zero integer is 1. Why? It should be clear that there is in general no integer k such that $a = 0 \cdot k$.

How do we find all divisors of a given integer? From the prime number theorem, we know that x is uniquely represented by the product of its prime factors. Every divisor is the product of some subset of these prime factors. Such subsets can be constructed using backtracking techniques as discussed in Chapter 8, but we must be careful about duplicate prime factors. For example, the prime factorization of 12 has three terms (2, 2, and 3) but 12 has only 6 divisors (1, 2, 3, 4, 6, 12).

7.2.1 Greatest Common Divisor

Since 1 divides every integer, the least common divisor of every pair of integers a, b is 1. Far more interesting is the *greatest common divisor*, or *gcd*, the *largest* divisor shared by a given pair of integers. Consider a fraction x/y, say, 24/36. The reduced form of this fraction comes after we divide both the numerator and denominator by $gcd(x, y)$, in this case 12. We say two integers are *relatively prime* if their greatest common divisor is 1.

Euclid's algorithm for finding the greatest common divisor of two integers has been called history's first interesting algorithm. The naive way to compute gcd would be to test all divisors of the first integer explicitly on the second, or perhaps to find the prime factorization of both integers and take the product of all factors in common. But both approaches involve computationally intensive operations.

Euclid's algorithm rests on two observations. First,

> If $b|a$, then $gcd(a, b) = b$.

This should be pretty clear. If b divides a, then $a = bk$ for some integer k, and thus $gcd(bk, b) = b$. Second,

> If $a = bt + r$ for integers t and r, then $gcd(a, b) = gcd(b, r)$.

Why? By definition, $gcd(a, b) = gcd(bt + r, b)$. Any common divisor of a and b must rest totally with r, because bt clearly must be divisible by any divisor of b.

Euclid's algorithm is recursive, repeated replacing the bigger integer by its remainder mod the smaller integer. This typically cuts one of the arguments down by about half, and so after a logarithmic number of iterations gets down to the base case. Consider the following example. Let $a = 34398$ and $b = 2132$.

$$\begin{aligned} gcd(34398, 2132) &= gcd(34398 \bmod 2132, 2132) = gcd(2132, 286) \\ gcd(2132, 286) &= gcd(2132 \bmod 286, 286) = gcd(286, 130) \\ gcd(286, 130) &= gcd(286 \bmod 130, 130) = gcd(130, 26) \\ gcd(130, 26) &= gcd(130 \bmod 26, 26) = gcd(26, 0) \end{aligned}$$

Therefore, $gcd(34398, 2132) = 26$.

However, Euclid's algorithm can give us more than just the $gcd(a, b)$. It can also find integers x and y such that

$$a \cdot x + b \cdot y = gcd(a, b)$$

which will prove quite useful in solving linear congruences. We know that $gcd(a, b) = gcd(b, a')$, where $a' = a - b\lfloor a/b \rfloor$. Further, assume we know integers x' and y' such that

$$b \cdot x' + a' \cdot y' = gcd(a, b)$$

by recursion. Substituting our formula for a' into the above expression gives us

$$b \cdot x' + (a - b\lfloor a/b \rfloor) \cdot y' = gcd(a, b)$$

and rearranging the terms will give us our desired x and y. We need a basis case to complete our algorithm, but that is easy since $a \cdot 1 + 0 \cdot 0 = gcd(a, 0)$.

For the previous example, we get that $34398 \times 15 + 2132 \times -242 = 26$. An implementation of this algorithm follows below:

```
/*      Find the gcd(p,q) and x,y such that p*x + q*y = gcd(p,q)      */

long gcd(long p, long q, long *x, long *y)
{
        long x1,y1;                     /* previous coefficients */
        long g;                         /* value of gcd(p,q) */

        if (q > p) return(gcd(q,p,y,x));

        if (q == 0) {
                *x = 1;
                *y = 0;
                return(p);
        }

        g = gcd(q, p%q, &x1, &y1);

        *x = y1;
        *y = (x1 - floor(p/q)*y1);

        return(g);
}
```

7.2.2 Least Common Multiple

Another useful function on two integers is the *least common multiple* (lcm), the *smallest* integer which is divided by both of a given pair of integers. For example, the least common multiple of 24 and 36 is 72.

Least common multiple arises when we want to compute the simultaneous periodicity of two distinct periodic events. When is the next year (after 2000) that the presidential

election (which happens every 4 years) will coincide with census (which happens every 10 years)? The events coincide every twenty years, because $lcm(4, 10) = 20$.

It is self-evident that $lcm(x, y) \geq \max(x, y)$. Similarly, since $x \cdot y$ is a multiple of both x and y, $lcm(x, y) \leq xy$. The only way that there can be a smaller common multiple is if there is some non-trivial factor shared between x and y.

This observation, coupled with Euclid's algorithm, gives an efficient way to compute least common multiple, namely, $lcm(x, y) = xy/gcd(x, y)$. A slicker algorithm appears in [Dij76], which avoids the multiplication and hence possibility of overflow.

7.3 Modular Arithmetic

In Chapter 5, we reviewed the basic arithmetic algorithms for integers, such as addition and multiplication. We are not always interested in the full answers, however. Sometimes the remainder suffices for our purposes. For example, suppose your birthday this year falls on a Wednesday. What day of the week will it it will fall on next year? All you need to know is the remainder of the number of days between now and then (either 365 or 366) when dividing by the 7 days of the week. Thus it will fall on Wednesday plus one (365 mod 7) or two (366 mod 7) days, i.e., Thursday or Friday depending upon whether it is affected by a leap year.

The key to such efficient computations is *modular arithmetic*. Of course, we can in principle explicitly compute the entire number and then find the remainder. But for large enough integers, it can be much easier to just work with remainders via modular arithmetic.

The number we are dividing by is called the *modulus*, and the remainder left over is called the *residue*. The key to efficient modular arithmetic is understanding how the basic operations of addition, subtraction, and multiplication work over a given modulus:

- *Addition* — What is $(x + y) \bmod n$? We can simplify this to

$$((x \bmod n) + (y \bmod n)) \bmod n$$

to avoid adding big numbers. How much small change will I have if given \$123.45 by my mother and \$94.67 by my father?

$$(12,345 \bmod 100) + (9,467 \bmod 100) = (45 + 67) \bmod 100 = 12 \bmod 100$$

- *Subtraction* — Subtraction is just addition with negative values. How much small change will I have after spending \$52.53?

$$(12 \bmod 100) - (53 \bmod 100) = -41 \bmod 100 = 59 \bmod 100$$

Notice how we can convert a negative number mod n to a positive number by adding a multiple of n to it. Further, this answer makes sense in this change example. It is usually best to keep the residue between 0 and $n - 1$ to ensure we are working with the smallest-magnitude numbers possible.

- *Multiplication* — Since multiplication is just repeated addition,

$$xy \bmod n = (x \bmod n)(y \bmod n) \bmod n$$

How much change will you have if you earn $17.28 per hour for 2,143 hours?

$$(1,728 \times 2,143) \bmod 100 = (28 \bmod 100) \times (43 \bmod 100) = 4 \bmod 100$$

Further, since exponentiation is just repeated multiplication,

$$x^y \bmod n = (x \bmod n)^y \bmod n$$

Since exponentiation is the quickest way to produce really large integers, this is where modular arithmetic really proves its worth.

- *Division* — Division proves considerably more complicated to deal with, and will be discussed in Section 7.4.

Modular arithmetic has many interesting applications, including:

- *Finding the Last Digit* — What is the last digit of 2^{100}? Sure we can use infinite precision arithmetic and look at the last digit, but why? We can do this computation by hand. What we really want to know is what $2^{100} \bmod 10$ is. By doing repeated squaring, and taking the remainder $\bmod 10$ at each step we make progress very quickly:

$$\begin{aligned}
2^3 \bmod 10 &= 8 \\
2^6 \bmod 10 &= 8 \times 8 \bmod 10 \to 4 \\
2^{12} \bmod 10 &= 4 \times 4 \bmod 10 \to 6 \\
2^{24} \bmod 10 &= 6 \times 6 \bmod 10 \to 6 \\
2^{48} \bmod 10 &= 6 \times 6 \bmod 10 \to 6 \\
2^{96} \bmod 10 &= 6 \times 6 \bmod 10 \to 6 \\
2^{100} \bmod 10 &= 2^{96} \times 2^3 \times 2^1 \bmod 10 \to 6
\end{aligned}$$

- *RSA Encryption Algorithm* — A classic application of modular arithmetic on large integers arises in public-key cryptography, namely, the RSA algorithm. Here, our message is encrypted by coding it as an integer m, raising it to a power k, where k is the so-called public-key or encryption key, and taking the results mod n. Since m, n, and k are all huge integers, computing $m^k \bmod n$ efficiently requires the tools we developed above.

- *Calendrical Calculations* — As demonstrated with the birthday example, computing the day of the week a certain number of days from today, or the time a certain number of seconds from now, are both applications of modular arithmetic.